

الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

قسم هندسة البرمجيات ونظم المعلومات

مشروع مادة المترجمات — العام الدراسي 2026/2025

مترجم متعدد اللغات باستخدام ANTLR

ANTLR Multi-Language Compiler

إعداد

— أسماء الطلاب —

إشراف

— اسم المدرّس —

آب - 2026

6.....	تقرير مشروع ANTLR Multi–Language Compiler
6.....	مقدمة
.....	شو بيميز هاد المشروع؟
.....	خطأ! الإشارة المرجعية غير معروفة.
.....	التقنيات المستخدمة
.....	خطأ! الإشارة المرجعية غير معروفة.
6.....	بنية المشروع
7.....	شرح المجلدات الرئيسية
8.....	مراحل الترجمة (Compilation Pipeline)
8.....	المرحلة 1: التحليل المعجمي (Lexical Analysis)
9.....	المرحلة 2: التحليل النحوي (Syntax Analysis)
10.....	المرحلة 3: بناء AST (Abstract Syntax Tree)
10.....	المرحلة 4: التحليل الدلالي (Semantic Analysis)
11.....	المرحلة 5: العرض (Output)
12.....	مخطط بنية AST – Python
14.....	تفصيل عقد Python AST
16.....	مخطط بنية AST – Jinja2
17.....	تفصيل عقد Jinja2 AST
22.....	شرح القواعد (Grammars)
22.....	Python Grammar
25.....	Jinja2 Grammar
28.....	نمط الزائر (Visitor Pattern)
28.....	لشيش الـ Visitor Pattern؟
28.....	البنية الأساسية
29.....	ASTBuilder.java – بناء AST لـ Python
31.....	JinjaASTBuilder.java – بناء AST لـ Jinja2
33.....	ASTPrinter.java – طباعة AST بألوان
34.....	جدول الرموز (Symbol Table)
34.....	البنية الأساسية

34.....	النطاقات المتداخلة (Nested Scopes)
34.....	العمليات الأساسية
35.....	أنواع الرموز
36.....	مثال كامل
36.....	طباعة جدول الرموز
37.....	مثال تطبيقي - ترجمة كاملة
37.....	الكود المصدر
37.....	المرحلة 1: التحليل المعجمي
38.....	المرحلة 2: التحليل النحوي
39.....	المرحلة 3: بناء AST
39.....	المرحلة 4: التحليل الدلالي
40.....	المرحلة 5: العرض
41.....	الملفات الأساسية
43.....	ملفات Python AST (مختارة)
43.....	ملفات Jinja2 AST (مختارة)
44.....	ملفات CSS AST (مختارة)
45.....	مميزات متقدمة
45.....	1. List Comprehensions (Python)
45.....	2. Function Parameters المتنوعة
45.....	3. Jinja2 Whitespace Control
46.....	4. Template Inheritance
46.....	5. Macros مع معاملات
46.....	6. Filters
46.....	7. Tests
47.....	المرحلة الثانية: توليد الكود (Code Generation)
47.....	شو يعني "توليد كود"؟
47.....	المخطط العام للمرحلة الثانية
48.....	مراحل التوليد الخمسة

50.....	المخرجات النهائية
50.....	التوليد المتزايد (Incremental Generation)
50.....	قيم الـ Runtime
51.....	التحليل الدلالي متعدد المستويات
51.....	1. مستوى Python – PythonSemanticAnalyzer
51.....	2. مستوى Jinja – JinjaSemanticAnalyzer
51.....	3. مستوى المشروع – FlaskProjectAnalyzer
52.....	أمثلة على أخطاء دلالية مكتشفة
52.....	الخادم المباشر (Live Server)
52.....	كيف يشتغل؟
52.....	إعادة التحميل التلقائي
53.....	كيفية التشغيل
53.....	المتطلبات
53.....	خطوات التشغيل
54.....	شرح خيارات سطر الأوامر
54.....	إعادة توليد الـ Parsers (لو عدلنا القواعد)
54.....	ملفات الاختبار
55.....	الخلاصة
55.....	المرحلة الأولى – التحليل والفحص
55.....	المرحلة الثانية – توليد الكود

14.....	جدول 1 : عقد الجمل (Statement Nodes) ◆
14.....	جدول 2 : عقد التعابير - القيم الثابتة (Literals) ◆
15.....	جدول 3 : عقد التعابير - العمليات (Operations) ◆
15.....	جدول 4 : عقد التعابير - الوصول (Access) ◆
15.....	جدول 5 : عقد متقدمة ◆
15.....	جدول 6 : عقد معاملات الدوال (Parameter Nodes) ◆
17.....	جدول 7 : عقد المحتوى (Content Nodes) ◆
19.....	جدول 8 : عقد البلوكات (Block Nodes) ◆
19.....	جدول 9 : عقد الجمل (Statement Nodes) ◆
41.....	جدول 10 : الملفات الأساسية.....
43.....	جدول 11 : ملفات Python AST (مختارة).....
43.....	جدول 12 : ملفات Jinja2 AST (مختارة).....
44.....	جدول 13 : ملفات CSS AST (مختارة).....
51.....	جدول 14 : 1. مستوى Python - PythonSemanticAnalyzer.....
52.....	جدول 15 : أمثلة على أخطاء دلالية مكتشفة.....
54.....	جدول 16 : شرح خيارات سطر الأوامر.....
54.....	جدول 17 : ملفات الاختبار.....

تقرير مشروع ANTLR Multi-Language Compiler

مقدمة

هاد المشروع هو مترجم (compiler) تعليمي متقدم، مبني باستخدام ANTLR 4.13.2. المشروع يوضح كيف بنيني مترجم كامل من الصفر، ابتداءً من التحليل المعجمي ووصولاً للتحليل الدلالي وبناء شجرة AST.

بنية المشروع

المشروع منظم بطريقة احترافية، كل قسم إلى وظيفة محددة:

```
antlr_course/
├── grammars/                                # ملفات القواعد النحوية (.g4)
│   ├── pythonLexer.g4                      # Python المحلل المعجمي لـ
│   ├── pythonParser.g4                    # Python المحلل النحوي لـ
│   ├── jinja2Lexer.g4                     # Jinja2 المحلل المعجمي لـ
│   └── jinja2Parser.g4                    # Jinja2 المحلل النحوي لـ
├── src/
│   ├── Main.java                          # نقطة الدخول الرئيسية
│   └── antlr/
│       ├── ast/                           # (كلاس 136+) AST عقد شجرة
│       │   ├── node/                      # ASTNode الكلاس الأساسي
│       │   ├── python/                   # Python (~47) عقد
│       │   ├── jinja2/                   # Jinja2 (~60) عقد
│       │   └── css/                      # CSS (~30) عقد
│       ├── visitor/                       # تطبيقات نمط الزائر
│       │   ├── ASTBuilder.java            # Python لـ AST بناء
│       │   ├── JinjaASTBuilder.java       # Jinja2 لـ AST بناء
│       │   └── ASTPrinter.java            # AST طباعة
│       ├── gen/                           # ANTLR الكود المولد من
│       │   ├── python/
│       │   └── jinja2/
│       └── symbol/                        # التحليل الدلالي
│           ├── Symbol.java                # كلاس الرمز
│           └── SymbolTable.java           # جدول الرموز
├── tests/                                  # ملفات الاختبار
│   ├── dictionary.py                      # Python اختبار شامل لـ
│   └── flask/                             # Flask + Jinja2 مثال
│       ├── app.py
│       └── templates/
├── dependencies/
│   └── antlr-4.13.2-complete.jar
└── antlr-denter-main/                     # DenterHelper مكتبة
```

[/grammars](#)

هون بنحط ملفات g4 . اللي بتعرّف قواعد اللغة. كل لغة بدها ملفين:

- Lexer – بيحدد كيف نقسم النص لـ tokens (كلمات، رموز، أرقام)
- Parser – بيحدد كيف نركب هالـ tokens لتكوين جمل صحيحة

[/src/antlr/ast](#)

هون موجودة كل كلاسات عقد الـ AST. في عندنا +136 كلاس موزعين على:

- /python – عقد خاصة بلغة Python
- /jinja2 – عقد خاصة بـ Jinja2 وHTML
- /css – عقد خاصة بـ CSS

[/src/antlr/visitor](#)

الـ Visitors هن المسؤولين عن:

- تحويل Parse Tree لـ AST
- طباعة الـ AST بشكل مقروء

[/src/antlr/symbol](#)

هون موجود جدول الرموز (Symbol Table) اللي بيتتبع:

- المتغيرات وين تم تعريفها
- الدوال والـ classes
- النطاقات (Scopes) المتداخلة

مراحل الترجمة (Compilation Pipeline)

المرجم يشتغل على 5 مراحل متتالية، كل مرحلة بتحول الكود لشكل أسهل للمعالجة:

flowchart TD

```
A[Source Code<br/>المصدر] --> B[Phase 1: Lexical Analysis<br/>التحليل المعجمي]
B --> C[Phase 2: Syntax Analysis<br/>التحليل النحوي]
C --> D[Phase 3: AST Building<br/>بناء الشجرة المجردة]
D --> E[Phase 4: Semantic Analysis<br/>التحليل الدلالي]
E --> F[Phase 5: Output<br/>العرض والطباعة]

B -->|TokenStream| C
C -->|Parse Tree| D
D -->|AST| E
E -->|Symbol Table| F
```

المرحلة 1: التحليل المعجمي (Lexical Analysis)

شو بيصير؟

- بنأخذ النص العادي ونقسمو لـ tokens (وحدات معجمية)
- كل token بيمثل كلمة محجوزة، رقم، رمز، أو اسم متغير

مثال:

```
x = 10 + 5
```

بيصير:

[NAME]	'x'	(Line 1, Col 0)
[ASSIGN]	'='	(Line 1, Col 2)
[NUM]	'10'	(Line 1, Col 4)
[PLUS]	'+'	(Line 1, Col 7)
[NUM]	'5'	(Line 1, Col 9)

ميزة خاصة لـ Python:

المحلل المعجمي بيستخدم DenterHelper لتوليد tokens وهمية INDENT و DEDENT بناءً على المسافات البادئة. هيك بنقدر نفهم البلوكات (blocks) بدون أقواس.

ميزة خاصة لـ Jinja2:

المحلل فيه 7 أنماط (modes) مختلفة يبتدل بينهم حسب السياق:

- DEFAULT_MODE - محتوى HTML عادي
- HTML_TAG - داخل tag مثل <div ...>
- STYLE_TAG - داخل <style> attributes
- CSS_CONTENT - محتوى CSS داخل <style>...</style>
- JINJA_STMT - داخل {% ... %}
- JINJA_EXPR - داخل {{ ... }}
- JINJA_COMMENT - داخل {# ... #}

المرحلة 2: التحليل النحوي (Syntax Analysis)

شو بيصير؟

- بناخد الـ tokens ونتأكد إنهم متطابقين مع قواعد اللغة
- بنبني Parse Tree (شجرة نحوية) بتمثل بنية الجملة

مثال:

```
if x > 5:
    print(x)
```

:Parse Tree

```
IfStatementContext
├── IF: 'if'
├── ComparisonContext
│   ├── NAME: 'x'
│   ├── GT: '>'
│   └── NUM: '5'
├── COLON: ':'
└── BlockContext
    └── PrintStatementContext
        └── NAME: 'x'
```

كشف الأخطاء:

لو كان في خطأ نحوي (مثلاً نسينا : بعد if)، المرحلة هاي بتكتشفو ويتوقف الترجمة.

المرحلة 3: بناء AST (Abstract Syntax Tree)

شو بيصير؟

- بنحول Parse Tree (المفصلة) لـ AST (مختصرة)
- الـ AST بتحتفظ بس بالمعلومات المهمة (بدون أقواس، نقاط فاصلة، الخ)

كيف بيصير التحويل؟

بنستخدم Visitor Pattern - الـ ASTBuilder بيمشي على Parse Tree وينشئ عقد AST:

```
public ASTNode visitIfStmt(IfStmtContext ctx) {
    // نستخرج الشرط
    ExpressionNode condition = (ExpressionNode) visit(ctx.expression());

    // نستخرج البلك
    BlockNode block = (BlockNode) visit(ctx.block());

    // ننشئ عقدة if
    return new IfStatementNode(condition, block, ctx.getStart().getLine());
}
```

النتيجة (AST):

```
IfStatementNode [line 1]
├── condition: ComparisonNode (>)
│   ├── VariableNode: x
│   └── NumberIntegerNode: 5
└── BlockNode
    ├── PrintNode
    └── VariableNode: x
```

المرحلة 4: التحليل الدلالي (Semantic Analysis)

شو بيصير؟ (بس لـ Python)

- بنبني Symbol Table - جدول بكل المتغيرات والدوال
- بنتبع النطاقات (Scopes) - كل دالة إلها نطاقها الخاص
- بنتأكد من إنا المتغيرات معرفة قبل استخدامها

مثال على جدول الرموز:

لو عندنا كود:

```
def calculate(x, y):
    result = x + y
    return result

total = calculate(10, 5)
```

جدول الرموز:

Global Scope (level 0)			
Name	Type	Line	Col
calculate	FUNCTION	1	4
total	VARIABLE	5	0
function_calculate (level 1)			
x	PARAMETER	1	14
y	PARAMETER	1	17
result	VARIABLE	2	4

ليش هالشى مهم؟

- لو استخدمنا متغير قبل ما نعرفو، جدول الرموز بيكتشف الخطأ
- بيساعدنا نفهم وين كل متغير متعرف ووين بنقدر نستخدمو

المرحلة 5: العرض (Output)

شو بيصير؟

- بنطبع الـ AST بشكل شجري ملون
- بنطبع جدول الرموز بشكل جدول منظم
- بنستخدم ANSI colors عشان يكون العرض واضح

مثال الخرج:

```
Abstract Syntax Tree:
ProgramNode
├── FunctionDefinitionNode [line 1]
│   ├── name: calculate
│   ├── RegularParamNode: x
│   ├── RegularParamNode: y
│   └── BlockNode
│       ├── AssignmentNode [line 2]
│       │   ├── target: result
│       │   └── BinaryOpNode (+)
│       ├── ReturnNode
│       │   └── VariableNode: result
│       └── AssignmentNode [line 5]
│           ├── target: total
│           └── FunctionCallNode: calculate
```

الـ AST يتكون من عقد (nodes) منظمة بشكل هرمي. كل عقدة يتمثل جزء من الكود.

```
classDiagram
class ASTNode {
    <<abstract>>
    +String nodeName
    +int lineNumber
    +int columnNumber
    +accept(visitor)
    +getChildren()
}

class ProgramNode {
    +List~StatementNode~ statements
}

class StatementNode {
    <<abstract>>
}

class ExpressionNode {
    <<abstract>>
}

class AssignmentNode {
    +String targetName
    +ExpressionNode value
}

class IfStatementNode {
    +ExpressionNode condition
    +BlockNode thenBlock
    +BlockNode elseBlock
}

class ForStatementNode {
    +String variable
    +ExpressionNode iterable
    +BlockNode body
}

class WhileStatementNode {
    +ExpressionNode condition
    +BlockNode body
}

class FunctionDefinitionNode {
    +String name
    +List~ParameterNode~ parameters
    +BlockNode body
}

class ClassDefinitionNode {
    +String name
    +String parentClass
}
```

```

    +BlockNode body
}

class NumberIntegerNode {
    +int value
}

class StringNode {
    +String value
}

class ListNode {
    +List~ExpressionNode~ elements
}

class BinaryOpNode {
    +Operator op
    +ExpressionNode left
    +ExpressionNode right
}

class VariableNode {
    +String name
}

class FunctionCallNode {
    +String functionName
    +List~ExpressionNode~ arguments
}

ASTNode <|-- ProgramNode
ASTNode <|-- StatementNode
ASTNode <|-- ExpressionNode

StatementNode <|-- AssignmentNode
StatementNode <|-- IfStatementNode
StatementNode <|-- ForStatementNode
StatementNode <|-- WhileStatementNode
StatementNode <|-- FunctionDefinitionNode
StatementNode <|-- ClassDefinitionNode

ExpressionNode <|-- NumberIntegerNode
ExpressionNode <|-- StringNode
ExpressionNode <|-- ListNode
ExpressionNode <|-- BinaryOpNode
ExpressionNode <|-- VariableNode
ExpressionNode <|-- FunctionCallNode

```

تفصيل عقد Python AST

عقد الجمل (Statement Nodes)

جدول 1 : عقد الجمل (Statement Nodes)

العقدة	الوصف	مثال
AssignmentNode	إسناد قيمة لمتغير	x = 10
PrintNode	طباعة قيمة	print(x)
ReturnNode	إرجاع قيمة من دالة	return x + 1
IfStatementNode	جملة شرطية	if x > 5: ...
ForStatementNode	حلقة for	for i in range(10): ...
WhileStatementNode	حلقة while	while x > 0: ...
BreakNode	كسر حلقة	break
ContinueNode	متابعة حلقة	continue
FunctionDefinitionNode	تعريف دالة	def foo(x): ...
ClassDefinitionNode	تعريف class	class MyClass: ...
ImportStatement	استيراد	from math import sqrt

عقد التعابير - القيم الثابتة (Literals)

جدول 2 : عقد التعابير - القيم الثابتة (Literals)

العقدة	الوصف	مثال
NumberIntegerNode	عدد صحيح	42
NumberDoubleNode	عدد عشري	3.14
StringNode	نص	"hello"
BooleanNode	قيمة منطقية	False ,True
NoneNode	قيمة فارغة	None
ListNode	قائمة	[1, 2, 3]
DictNode	قاموس	{"key": "value"}

◆ عقد التعابير - العمليات (Operations)

جدول 3 : عقد التعابير - العمليات (Operations)

العقدة	الوصف	أمثلة
BinaryOpNode	عمليات ثنائية حسابية	+, -, *, /, %, //
UnaryOpNode	عمليات أحادية	-x, +x
ComparisonNode	عمليات مقارنة	not, in, >, <=, >, <, !=, ==
LogicalOpNode	عمليات منطقية	not, or, and

◆ عقد التعابير - الوصول (Access)

جدول 4 : عقد التعابير - الوصول (Access)

العقدة	الوصف	مثال
VariableNode	متغير	x
DotAccessNode	وصول بالنقطة	obj.attribute
IndexAccessNode	وصول بالفهرس	list[0]
FunctionCallNode	استدعاء دالة	foo(x, y)

◆ عقد متقدمة

جدول 5 : عقد متقدمة

العقدة	الوصف	مثال
ListComprehensionNode	list comprehension	[x*2 for x in range(10)]
ParenExprNode	تعبير بأقواس	(x + y) * z

◆ عقد معاملات الدوال (Parameter Nodes)

جدول 6 : عقد معاملات الدوال (Parameter Nodes)

العقدة	الوصف	مثال
RegularParamNode	معامل عادي	def foo(x, y=10)
StarArgsNode	معامل args*	def foo(*args)
KwargsNode	معامل kwargs**	def foo(**kwargs)
KeywordOnlyParamNode	معامل keyword-only	def foo(*, x, y)
PositionalOnlyParamNode	معامل positional-only	def foo(x, /, y)

قوالب Jinja2 أعقد من Python لأنها بتجمع 3 لغات بملف واحد: Jinja2 + HTML + CSS

```
classDiagram
    class ASTNode {
        <<abstract>>
    }

    class TemplateNode {
        +List~ASTNode~ content
    }

    class HtmlTextNode {
        +String text
    }

    class HtmlElementNode {
        <<abstract>>
        +String tagName
        +List~HtmlAttributeNode~ attributes
    }

    class NormalHtmlElementNode {
        +List~ASTNode~ children
    }

    class VoidHtmlElementNode {
    }

    class ForBlockNode {
        +TargetNode target
        +JinjaExpressionNode iterable
        +List~ASTNode~ body
        +List~ASTNode~ elseBody
    }

    class IfBlockNode {
        +JinjaExpressionNode condition
        +List~ASTNode~ thenBody
        +List~ElifBranchNode~ elifBranches
        +List~ASTNode~ elseBody
    }

    class MacroNode {
        +String name
        +List~MacroParamNode~ parameters
        +List~ASTNode~ body
    }

    class BlockDefinitionNode {
        +String name
        +List~ASTNode~ content
    }

    class ExpressionBlockNode {
        +JinjaExpressionNode expression
    }
```



```

+List~FilterNode~ filters
}

class SetStatementNode {
+TargetNode target
+JinjaExpressionNode value
}

class ImportNode {
+String template
+String alias
}

class CSSStyleSheetNode {
+List~CSSRuleNode~ rules
}

class CSSRuleNode {
+CSSSelectorNode selector
+List~CSSDeclarationNode~ declarations
}

ASTNode <|-- TemplateNode
ASTNode <|-- HtmlTextNode
ASTNode <|-- HtmlElementNode
ASTNode <|-- ForBlockNode
ASTNode <|-- IfBlockNode
ASTNode <|-- MacroNode
ASTNode <|-- BlockDefinitionNode
ASTNode <|-- ExpressionBlockNode
ASTNode <|-- SetStatementNode
ASTNode <|-- ImportNode
ASTNode <|-- CSSStyleSheetNode

HtmlElementNode <|-- NormalHtmlElementNode
HtmlElementNode <|-- VoidHtmlElementNode

```

تفصيل عقد AST Jinja2

عقد المحتوى (Content Nodes)

جدول 7 : عقد المحتوى (Content Nodes)

مثال	الوصف	العقدة
جذر الشجرة	القالب الكامل	TemplateNode
<p>Hello World</p> النص: "Hello" "World"	نص HTML عادي	HtmlTextNode
...	عنصر HTML	HtmlElementNode
class="container"	خاصية HTML	HtmlAttributeNode

المشروع يبدع عناصر HTML محددة كل واحد إلى كلاس خاص:

عناصر بنية المستند:

- `<html>` - `HtmlRootHtmlNode`
- `<head>` - `HeadHtmlNode`
- `<body>` - `BodyHtmlNode`
- `<title>` - `TitleHtmlNode`

عناصر النص والتنسيق:

- `<div>` - `DivHtmlNode`
- `<p>` - `ParagraphHtmlNode`
- `<h1>` - `H1HtmlNode`
- `<a>` - `AnchorHtmlNode`

عناصر النماذج:

- `<form>` - `FormHtmlNode`
- (void element) `<input>` - `InputHtmlNode`
- `<button>` - `ButtonHtmlNode`
- `<label>` - `LabelHtmlNode`
- `<textarea>` - `TextareaHtmlNode`

عناصر أخرى:

- (void element) `<img>` - `ImgHtmlNode`
- `<style>` - `StyleHtmlNode` (بيحتوي CSS)
- `GenericHtmlNode` - أي element ثاني

جدول 8 : عقد البلوكات (Block Nodes) 

العقدة	الوصف	مثال
ForBlockNode	حلقة for	{% for item in items %}...{% endfor %}
IfBlockNode	شرط if	{% if condition %}...{% elif %}...{% else %}...{% endif %}
MacroNode	تعريف macro	{% macro button(text) %}...{% endmacro %}
CallBlockNode	استدعاء macro	{% call button("Click") %}...{% endcall %}
BlockDefinitionNode	تعريف block	{% block content %}...{% endblock %}
WithBlockNode	نطاق with	{% with x = 10 %}...{% endwith %}
FilterBlockNode	تطبيق filter	{% filter upper %}...{% endfilter %}
AutoescapeBlockNode	تنعيل/تعطيل autoescape	{% autoescape true %}...{% endautoescape %}

جدول 9 : عقد الجمل (Statement Nodes) 

العقدة	الوصف	مثال
SetStatementNode	تعيين متغير	{% set x = 10 %}
SetBlockNode	تعيين بلوك	{% set x %}...{% endset %}
ImportNode	استيراد قالب	{% import "forms.html" as forms %}
FromImportNode	استيراد جزئي	{% from "forms.html" import input %}
IncludeNode	إدراج قالب	{% include "header.html" %}
ExtendsNode	امتداد قالب	{% extends "base.html" %}
DoStatementNode	تنفيذ تعبير	{% do list.append(1) %}

عقد التعابير (Expression Nodes)

التعابير بـ Jinja2 مشابهة لـ Python:

القيم الثابتة:

- JinjaNumberDoubleNode ,JinjaNumberIntegerNode ,JinjaStringNode
- JinjaNoneNode ,JinjaBooleanNode
- JinjaDictNode ,JinjaListNode

العمليات:

- JinjaBinaryOpNode - + , - , * , / , % , // , **
- JinjaUnaryOpNode - + , - , not
- JinjaComparisonNode - == , != , < , > , in , is
- JinjaLogicalOpNode - and , or

الوصول:

- VariableExprNode - variable_name
- DotAccessExprNode - obj.attr
- IndexAccessExprNode - list[0]
- FunctionCallExprNode - func(args)

متقدمة:

- TernaryExprNode - value if condition else other
- TestExprNode - var is none , var is defined
- FilterNode - value | filter(args)
- ParenExprNode - (expression)
- SliceExprNode - list[1:5]

عقد CSS

لما يكون في <style> tag، سيتم parsing لا CSS:

البنية:

- CSSStyleSheetNode - ورقة الأنماط الكاملة

• CSS (selector + declarations) قاعدة – CSSRuleNode

• (property: value) إعلان واحد – CSSDeclarationNode

:Selectors

• button ,p ,div – CSSElementSelectorNode

• .my-class – CSSClassSelectorNode

• div p – CSSDescendantSelectorNode (مسافة)

• :focus ,:hover – CSSPseudoClassSelectorNode

Properties المدعومة (+20 خاصية):

• color: red – ColorPropertyNode

• background: #fff – BackgroundPropertyNode

• font-family: Arial – FontFamilyPropertyNode

• font-weight: bold – FontWeightPropertyNode

• border: 1px solid black – BorderPropertyNode

• box-shadow: 2px 2px 4px rgba(0,0,0,0.2) – BoxShadowPropertyNode

• transform: translateY(-2px) – TransformPropertyNode

• transition: all 0.3s – TransitionPropertyNode

• height ,width ,padding ,margin – LengthPropertyNode

• الخ ,position ,display – KeywordPropertyNode

:Values

• rgb(255,0,0) ,#fff – CSSColorValueNode

• 50% ,2em ,10px – CSSLengthValueNode

• 0.5 ,1 – CSSNumberValueNode

• inherit ,none ,auto – CSSKeywordValueNode

• rgba() ,translateY() ,rgb() – CSSFunctionValueNode

◆ عقد الأهداف (Target Nodes)

بتستخدم بـ set, for, الخ:

- x – SimpleTargetNode
- x, y, z – TupleTargetNode
- namespace.attr – NamespaceTargetNode

شرح القواعد (Grammars)

Python Grammar

pythonLexer.g4 – المحلل المعجمي

DenterHelper – التعامل مع المسافات البادئة:

Python بتعتمد على المسافات البادئة لتحديد البلوكات. DenterHelper بيولد tokens وهمية:

```
private final DenterHelper denter = new DenterHelper(NL,
    pythonParser.INDENT,
    pythonParser.DEDENT)
{
    @Override
    public Token pullToken() {
        Token token = pythonLexer.super.nextToken();

        // Track bracket depth
        if (type == LPAREN || type == LBRACK || type == LBRACE) {
            bracketDepth++;
        }

        // Skip NL tokens inside brackets
        while (bracketDepth > 0 && token.getType() == NL) {
            token = pythonLexer.super.nextToken();
        }

        return token;
    }
};
```

مثال:

```
def foo():  
    x = 1  
    if True:  
        print(x)
```

Tokens المولدة:

```
DEF, NAME(foo), LPAREN, RPAREN, COLON, NL  
INDENT          # ← مولد تلقائياً  
NAME(x), ASSIGN, NUM(1), NL  
IF, TRUE, COLON, NL  
INDENT          # ← مولد تلقائياً  
PRINT, LPAREN, NAME(x), RPAREN, NL  
DEDENT          # ← مولد تلقائياً  
DEDENT          # ← مولد تلقائياً  
EOF
```

Tokens الأساسية:

```
NUM: [0-9]+;  
DOUBLE: [0-9]+ ('.' [0-9]*)?;  
STRING: '"' (~["\r\n])* '"' | '\'' (~['\r\n])* '\'';  
NAME: [a-zA-Z_][a-zA-Z0-9_]*;  
  
// Keywords  
IF: 'if';  
ELIF: 'elif';  
ELSE: 'else';  
FOR: 'for';  
WHILE: 'while';  
DEF: 'def';  
CLASS: 'class';  
RETURN: 'return';  
// ...
```

أولويات العمليات (من الأدنى للأعلى):

```

expression
    : ternary_expr
    ;

ternary_expr
    : or_expr ('if' or_expr 'else' ternary_expr)?    // أدنى أولوية
    ;

or_expr
    : and_expr ('or' and_expr)*
    ;

and_expr
    : not_expr ('and' not_expr)*
    ;

not_expr
    : 'not' not_expr
    | comparison
    ;

comparison
    : additive_expr (comp_op additive_expr)*
    ;

additive_expr
    : multiplicative_expr (('+' | '-') multiplicative_expr)*
    ;

multiplicative_expr
    : unary_expr (('*' | '/' | '%' | '//') unary_expr)*
    ;

unary_expr
    : ('+' | '-') unary_expr
    | power_expr
    ;

power_expr
    : postfix_expr ('**' unary_expr)?                // أعلى أولوية
    ;

postfix_expr
    : primary_expr (call | index | slice | dot)*
    ;

```


مثال على الأولوية:

```
result = 2 + 3 * 4 ** 2
```

ال Parser بينهما:

```
2 + (3 * (4 ** 2))
= 2 + (3 * 16)
= 2 + 48
= 50
```

Jinja2 Grammar

jinja2Lexer.g4 – المحلل المعجمي متعدد الأنماط

ال Lexer فيه 7 أنماط بيتبدل بينهم ديناميكياً:

1. DEFAULT_MODE – محتوى HTML:

```
STMT_OPEN      : '{%' -> pushMode(JINJA_STMT);
EXPR_OPEN      : '{{' -> pushMode(JINJA_EXPR);
COMMENT_OPEN   : '{#' -> pushMode(JINJA_COMMENT);
TAG_OPEN       : '<' -> pushMode(HTML_TAG);
HTML_TEXT      : ~[<{&]+ | '{' ~[%#{] | '<' ~[!a-zA-Z/];
```

2. HTML_TAG – داخل tag:

```
mode HTML_TAG;

TAG_STMT_OPEN  : '{%' -> pushMode(JINJA_STMT);
TAG_EXPR_OPEN  : '{{' -> pushMode(JINJA_EXPR);
TAG_CLOSE      : '>' -> popMode;
TAG_SELF_CLOSE : '/>' -> popMode;
TAG_DQUOTE     : '"' -> pushMode(ATTR_DQUOTE);
TAG_SQUOTE     : '\'' -> pushMode(ATTR_SQUOTE);

STYLE_TAG      : [Ss][Tt][Yy][Ll][Ee] -> mode(STYLE_TAG_MODE);
DIV_TAG        : [Dd][Ii][Vv];
P_TAG          : [Pp];
// ...
```

3. attributes – STYLE_TAG_MODE ال style:

```
mode STYLE_TAG_MODE;

STYLE_CLOSE    : '>' -> mode(CSS_CONTENT);
```

.4 CSS_CONTENT - محتوى CSS:

```
mode CSS_CONTENT;

STYLE_END      : '</' [Ss][Tt][Yy][Ll][Ee] '>' -> mode(DEFAULT_MODE);
CSS_SELECTOR    : ...;
CSS_PROPERTY    : ...;
```

.5 ATTR_DQUOTE - داخل "...":

```
mode ATTR_DQUOTE;

ATTR_DQ_CLOSE   : '"' -> popMode;
ATTR_DQ_EXPR    : '{{' -> pushMode(JINJA_EXPR);
ATTR_DQ_TEXT    : ~["{}+ | '{' ~[{}];
```

.6 JINJA_STMT - داخل {% %}:

```
mode JINJA_STMT;

STMT_CLOSE      : '%}' -> popMode;
STMT_CLOSE_TRIM : '%-}' -> popMode;

// Jinja2 keywords
FOR              : 'for';
IF              : 'if';
ELIF            : 'elif';
BLOCK           : 'block';
MACRO           : 'macro';
// ...
```

.7 JINJA_EXPR - داخل {{ }}:

```
mode JINJA_EXPR;

EXPR_CLOSE      : '}}' -> popMode;
EXPR_CLOSE_TRIM : '-}}' -> popMode;

// Expression tokens
PIPE            : '|';
DOT             : '.';
LBRACK          : '[';
// ...
```

مثال على التبديل:

```
<div class="{{ user.name }}">
  {% for item in items %}
    <p>{{ item }}</p>
  {% endfor %}
</div>
```

تسلسل الأنماط:

```
DEFAULT_MODE: <div
HTML_TAG: class=
ATTR_DQUOTE: "
JINJA_EXPR (push): {{ user.name }}
ATTR_DQUOTE (pop): "
HTML_TAG: >
DEFAULT_MODE (pop): \n
JINJA_STMT (push): {% for item in items %}
DEFAULT_MODE (pop): \n
HTML_TAG (push): <p>
DEFAULT_MODE (pop):
JINJA_EXPR (push): {{ item }}
DEFAULT_MODE (pop): </p>
JINJA_STMT (push): {% endfor %}
...
```

jinja2Parser.g4 – المحلل النحوي

القاعدة الرئيسية:

```
template
: templateContent* EOF
;

templateContent
: htmlElement          // <div>...</div>
| htmlText             // نص عادي
| expressionOutput     // {{ expr }}
| forBlock             // {% for %}...{% endfor %}
| ifBlock              // {% if %}...{% endif %}
| macroStatement       // {% macro %}...{% endmacro %}
| blockDefinition      // {% block %}...{% endblock %}
| setStatement         // {% set x = 1 %}
| importStatement      // {% import %}
| extendsStatement     // {% extends %}
// ...
;
```

مثال على قاعدة معقدة (for loop):

```
forBlock
: forStart templateContent* elseForBranch? forEnd
;

forStart
: STMT_OPEN FOR forTarget IN expression STMT_CLOSE
;

forTarget
: NAME                // for x in ...
| NAME (',' NAME)+    // for x, y in ...
;

elseForBranch
: STMT_OPEN ELSE STMT_CLOSE templateContent*
```

```

;
forEnd
: STMT_OPEN ENDFOR STMT_CLOSE
;

```

نمط الزائر (Visitor Pattern)

المشروع يستخدم Visitor Pattern - واحد من الأنماط التصميمية الكلاسيكية (Gang of Four Design Patterns).

ليش الـ Visitor Pattern؟

لما يكون عندنا شجرة (AST) فيها أنواع كتير من العقد، وبدنا نعمل عمليات مختلفة عليها:

- بدل ما نحط كل العمليات بكل عقدة (مسؤولية واحدة لكل كلاس)
- بنفصل العمليات ببراء بـ كلاسات Visitor منفصلة

البنية الأساسية

ASTNode.java - الكلاس الأساسي

```

public abstract class ASTNode {
    protected final String nodeName;
    protected final int lineNumber;
    protected final int columnNumber;
    protected ASTNode parent;

    // كل عقدة بتقبل زائر
    public abstract <T> T accept(ASTVisitor<T> visitor);

    // كل عقدة بتعطي أولادها
    public abstract List<ASTNode> getChildren();
}

```

ASTVisitor.java - الواجهة

```

public interface ASTVisitor<T> {
    // Python nodes
    T visit(ProgramNode node);
    T visit(AssignmentNode node);
    T visit(IfStatementNode node);
    T visit(ForStatementNode node);
    T visit(FunctionDefinitionNode node);
    T visit(NumberIntegerNode node);
    T visit(BinaryOpNode node);
    // ... 70+ visit methods

    // Jinja2 nodes
    T visit(TemplateNode node);
    T visit(ForBlockNode node);
    T visit(IfBlockNode node);
    T visit(HtmlElementNode node);
}

```

```
// ...  
}
```

ASTBuilder.java – بناء AST لـ Python

هاد الـ Visitor بيحول Parse Tree (من ANTLR) لـ AST (عقدنا).

مثال – تحويل If Statement:

```
@Override  
public ASTNode visitIfStmt(pythonParser.IfStmtContext ctx) {  
    int line = ctx.getStart().getLine();  
  
    // زيارة الشرط  
    ExpressionNode condition = (ExpressionNode) visit(ctx.test());  
  
    // زيارة البلاك الرئيسي  
    BlockNode thenBlock = new BlockNode(line);  
    for (pythonParser.StatementContext stmtCtx : ctx.block().statement()) {  
        StatementNode stmt = (StatementNode) visit(stmtCtx);  
        thenBlock.addStatement(stmt);  
    }  
  
    // زيارة elif branches  
    List<ElifBranchNode> elifBranches = new ArrayList<>();  
    for (pythonParser.ElifStmtContext elifCtx : ctx.elifStmt()) {  
        ExpressionNode elifCondition = (ExpressionNode) visit(elifCtx.test());  
        BlockNode elifBlock = (BlockNode) visit(elifCtx.block());  
        elifBranches.add(new ElifBranchNode(elifCondition, elifBlock,  
elifCtx.getStart().getLine()));  
    }  
  
    // زيارة else branch  
    BlockNode elseBlock = null;  
    if (ctx.elseStmt() != null) {  
        elseBlock = (BlockNode) visit(ctx.elseStmt().block());  
    }  
  
    return new IfStatementNode(condition, thenBlock, elifBranches, elseBlock, line);  
}
```

مثال – بناء Binary Operation:

```
@Override  
public ASTNode visitAddExpr(pythonParser.AddExprContext ctx) {  
    // operand لو في عملية واحدة بس، إرجع الـ  
    if (ctx.term().size() == 1) {  
        return visit(ctx.term(0));  
    }  
  
    // (left-associative) بناء سلسلة من العمليات  
    ExpressionNode result = (ExpressionNode) visit(ctx.term(0));  
  
    for (int i = 1; i < ctx.term().size(); i++) {  
        String op = ctx.getChild(2 * i - 1).getText(); // '+' or '-'  
        ExpressionNode right = (ExpressionNode) visit(ctx.term(i));
```

```

        BinaryOpNode.Operator operator = op.equals("+")
            ? BinaryOpNode.Operator.ADD
            : BinaryOpNode.Operator.SUB;

        result = new BinaryOpNode(operator, result, right, ctx.getStart().getLine());
    }

    return result;
}

```

:Symbol Table التعامل مع

```

@Override
public ASTNode visitFuncDef(pythonParser.FuncDefContext ctx) {
    String functionName = ctx.NAME().getText();
    int line = ctx.getStart().getLine();

    // تسجيل الدالة بجدول الرموز العام
    currentScope.define(functionName, Symbol.SymbolType.FUNCTION, line,
        ctx.NAME().getSymbol().getCharPositionInLine());

    // دخول نطاق جديد للدالة
    currentScope = currentScope.enterScope("function_" + functionName);

    // معالجة المعاملات
    List<ParameterNode> parameters = new ArrayList<>();
    if (ctx.params() != null) {
        parameters = visitParameters(ctx.params());
    }

    // معالجة جسم الدالة
    BlockNode body = (BlockNode) visit(ctx.block());

    // الخروج من نطاق الدالة
    currentScope = currentScope.exitScope();

    return new FunctionDefinitionNode(functionName, parameters, body, line);
}

```

أعقد من Python لأنو بيتعامل مع HTML, CSS, وJinja2 بنفس الوقت.

مثال – معالجة HTML Element:

```
@Override
public ASTNode visitDivElement(jinja2Parser.DivElementContext ctx) {
    int line = ctx.getStart().getLine();

    // جمع attributes
    List<HtmlAttributeNode> attributes = new ArrayList<>();
    for (jinja2Parser.HtmlAttributeContext attrCtx : ctx.htmlAttribute()) {
        attributes.add((HtmlAttributeNode) visit(attrCtx));
    }

    // جمع الأولاد (children)
    List<ASTNode> children = new ArrayList<>();
    for (jinja2Parser.HtmlContentContext contentCtx : ctx.htmlContent()) {
        ASTNode child = visit(contentCtx);
        if (child != null) {
            children.add(child);
        }
    }

    return new DivHtmlNode(attributes, children, line);
}
```

مثال – معالجة Attribute مع Jinja2:

```
@Override
public ASTNode visitAttrDqContent(jinja2Parser.AttrDqContentContext ctx) {
    if (ctx.attrDqText() != null) {
        // نص عادي
        return new HtmlTextNode(ctx.attrDqText().getText(), ctx.getStart().getLine());
    } else if (ctx.attrDqExpr() != null) {
        // Jinja2: {{ expr }}
        return visit(ctx.attrDqExpr().expression());
    } else if (ctx.attrDqStmt() != null) {
        // Jinja2: {% stmt %}
        return visit(ctx.attrDqStmt());
    }
    return null;
}
```

مثال – معالجة For Loop:

```
@Override
public ASTNode visitForLoopBlock(jinja2Parser.ForLoopBlockContext ctx) {
    int line = ctx.forStart().getStart().getLine();

    // المتغير (target)
    TargetNode target = (TargetNode) visit(ctx.forStart().forTarget());

    // iterable
```

```

    JinjaExpressionNode iterable = (JinjaExpressionNode)
visit(ctx.forStart().expression());

    // جسم الحلقة
    List<ASTNode> body = new ArrayList<>();
    for (jinja2Parser.TemplateContentContext contentCtx : ctx.templateContent()) {
        ASTNode child = visit(contentCtx);
        if (child != null) {
            body.add(child);
        }
    }

    // else branch (لو موجود)
    List<ASTNode> elseBody = new ArrayList<>();
    if (ctx.elseForBranch() != null) {
        for (jinja2Parser.TemplateContentContext contentCtx :
ctx.elseForBranch().templateContent()) {
            ASTNode child = visit(contentCtx);
            if (child != null) {
                elseBody.add(child);
            }
        }
    }

    return new ForBlockNode(target, iterable, body, elseBody, line);
}

```

مثال - معالجة CSS:

```

@Override
public ASTNode visitCssRule(jinja2Parser.CssRuleContext ctx) {
    // selector
    CSSSelectorNode selector = (CSSSelectorNode) visit(ctx.cssSelector());

    // declarations
    List<CSSDeclarationNode> declarations = new ArrayList<>();
    for (jinja2Parser.CssDeclarationContext declCtx : ctx.cssDeclaration()) {
        CSSDeclarationNode decl = (CSSDeclarationNode) visit(declCtx);
        if (decl != null) {
            declarations.add(decl);
        }
    }

    return new CSSRuleNode(selector, declarations, ctx.getStart().getLine());
}

```


هاد ال Visitor بياخد AST ويبطبعها بشكل شجري ملون بالـ console.

مثال – طباعة If Statement:

```
@Override
public String visit(IfStatementNode node) {
    StringBuilder sb = new StringBuilder();

    sb.append(indent()).append(TEAL).append("├─ ").append(LIGHT)
      .append("IfStatementNode").append(RESET)
      .append(GRAY).append(" [line
").append(node.getLineNumber()).append("]").append(RESET)
      .append("\n");

    // طباعة الشرط
    indentLevel++;
    sb.append(indent()).append(DARK).append("condition:").append(RESET).append("\n");
    indentLevel++;
    sb.append(node.getCondition().accept(this));
    indentLevel--;

    // طباعة then block
    sb.append(indent()).append(DARK).append("then:").append(RESET).append("\n");
    indentLevel++;
    sb.append(node.getThenBlock().accept(this));
    indentLevel--;

    // طباعة elif branches
    for (ElifBranchNode elif : node.getElifBranches()) {
        sb.append(elif.accept(this));
    }

    // طباعة else block
    if (node.getElseBlock() != null) {
        sb.append(indent()).append(DARK).append("else:").append(RESET).append("\n");
        indentLevel++;
        sb.append(node.getElseBlock().accept(this));
        indentLevel--;
    }

    indentLevel--;
    return sb.toString();
}
```

الألوان المستخدمة:

```
public static final String TEAL = "\u001B[38;2;0;173;181m"; // العناوين
public static final String LIGHT = "\u001B[38;2;238;238;238m"; // النص الرئيسي
public static final String GRAY = "\u001B[38;2;255;46;99m"; // المعلومات الإضافية
public static final String DARK = "\u001B[38;2;8;217;214m"; // Labels
```

جدول الرموز (Symbol Table)

جدول الرموز هو بنية بيانات تتبع كل المتغيرات والدوال المعرفة بالبرنامج.

البنية الأساسية

```
public class SymbolTable {
    private final String scopeName;           // اسم النطاق
    private final int scopeLevel;             // مستوى التداخل (0 = global)
    private final SymbolTable parent;         // النطاق الأب
    private final Map<String, Symbol> symbols; // الرموز بهاد النطاق
    private final List<SymbolTable> children; // النطاقات الفرعية
}
```

النطاقات المتداخلة (Nested Scopes)

كل دالة إليها نطاقها الخاص، والنطاقات تتنظم بشكل هرمي:

```
Global Scope (level 0)
├── function_calculate (level 1)
│   └── if_scope_1 (level 2)
└── function_main (level 1)
    └── for_scope_1 (level 2)
```

العمليات الأساسية

1. تعريف رمز (Define)

```
public boolean define(Symbol symbol) {
    if (symbols.containsKey(symbol.getName())) {
        return false; // الرمز موجود مسبقاً
    }
    symbols.put(symbol.getName(), symbol);
    return true;
}
```

مثال:

```
x = 10 # ← define("x", VARIABLE, line=1, col=0)
```

2. البحث عن رمز (Resolve)

```
public Symbol resolve(String name) {
    // البحث بالنطاق الحالي
    Symbol symbol = symbols.get(name);
    if (symbol != null) {
        return symbol;
    }

    // لو مش موجود، ابحث بالنطاق الأب
    if (parent != null) {
        return parent.resolve(name);
    }
}
```

```

    }

    return null; // مش موجود بأي نطاق
}

```

مثال:

```

def outer():
    x = 10
    def inner():
        print(x) # ← resolve("x") من بيطلع inner لـ outer

```

3. دخول نطاق جديد (Enter Scope)

```

public SymbolTable enterScope(String scopeName) {
    SymbolTable newScope = new SymbolTable(scopeName, scopeLevel + 1, this);
    children.add(newScope);
    return newScope;
}

```

4. الخروج من نطاق (Exit Scope)

```

public SymbolTable exitScope() {
    return parent;
}

```

أنواع الرموز

```

public enum SymbolType {
    VARIABLE,    // x = 10
    FUNCTION,    // def foo(): ...
    PARAMETER,   // def foo(x): ...
    CLASS        // class MyClass: ...
}

```

```

public void printAll() {
    printTable(0);
    for (SymbolTable child : children) {
        child.printAll();
    }
}

private void printTable(int indent) {
    String indentStr = " ".repeat(indent);

    System.out.println(indentStr + TEAL + "┌ " + scopeName + " (level " + scopeLevel
+ ")" + RESET);
    System.out.println(indentStr + TEAL + "|" + RESET);

    // رأس الجدول
    System.out.println(indentStr + TEAL + "|   Name           Type           Line   Col
Value" + RESET);
    System.out.println(indentStr + TEAL +
"|-----" + RESET);

    // الرموز
    for (Symbol symbol : symbols.values()) {
        System.out.printf(indentStr + TEAL + "|   " + LIGHT + "%-12s %-11s %-7d %-7d
%s" + RESET + "\n",
            symbol.getName(),
            symbol.getType(),
            symbol.getLine(),
            symbol.getColumn(),
            symbol.getValue() != null ? symbol.getValue() : "-");
    }

    System.out.println(indentStr + TEAL +
"|-----" + RESET);
}

```

مثال تطبيقي - ترجمة كاملة

خليني نوضح كيف بيشتغل المترجم من البداية للنهاية بمثال بسيط.

الكود المصدر

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

result = factorial(5)
print(result)
```

المرحلة 1: التحليل المعجمي

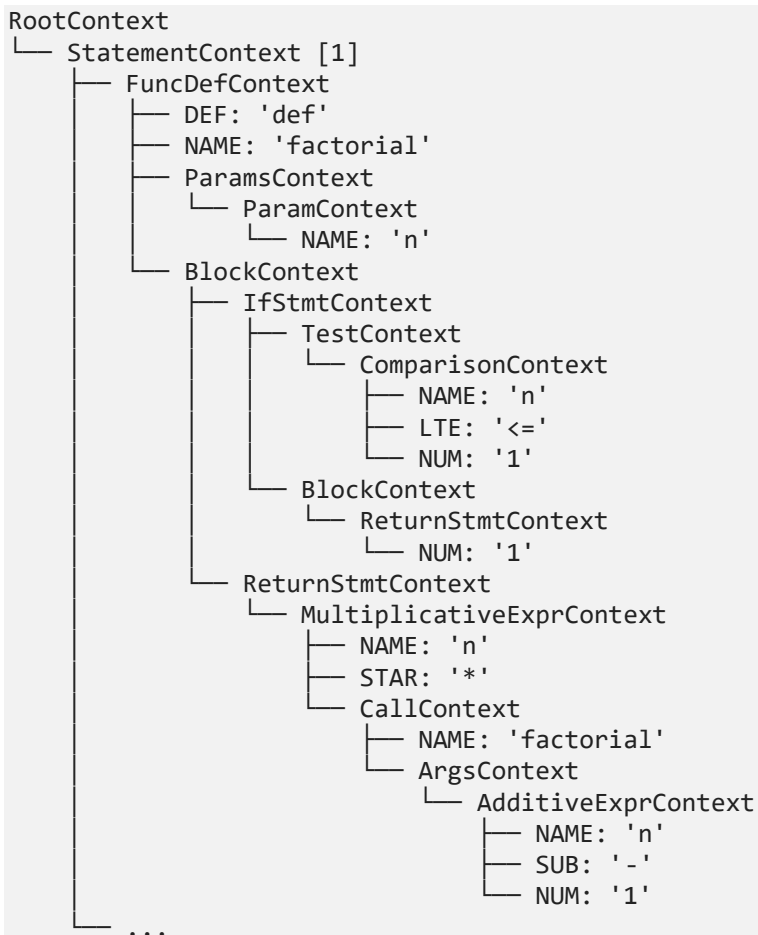
Tokens المولدة:

[DEF]	'def'	(Line 1, Col 0)
[NAME]	'factorial'	(Line 1, Col 4)
[LPAREN]	'('	(Line 1, Col 13)
[NAME]	'n'	(Line 1, Col 14)
[RPAREN]	)'	(Line 1, Col 15)
[COLON]	':'	(Line 1, Col 16)
[NL]	'newline'	(Line 1, Col 17)
[INDENT]	(generated)	(Line 2, Col 0)
[IF]	'if'	(Line 2, Col 4)
[NAME]	'n'	(Line 2, Col 7)
[LTE]	'<='	(Line 2, Col 9)
[NUM]	'1'	(Line 2, Col 12)
[COLON]	':'	(Line 2, Col 13)
[NL]	'newline'	(Line 2, Col 14)
[INDENT]	(generated)	(Line 3, Col 0)
[RETURN]	'return'	(Line 3, Col 8)
[NUM]	'1'	(Line 3, Col 15)
[NL]	'newline'	(Line 3, Col 16)
[DEDENT]	(generated)	(Line 4, Col 0)
[RETURN]	'return'	(Line 4, Col 4)
[NAME]	'n'	(Line 4, Col 11)
[STAR]	'*'	(Line 4, Col 13)
[NAME]	'factorial'	(Line 4, Col 15)
[LPAREN]	'('	(Line 4, Col 24)
[NAME]	'n'	(Line 4, Col 25)
[SUB]	'-'	(Line 4, Col 27)
[NUM]	'1'	(Line 4, Col 29)
[RPAREN]	)'	(Line 4, Col 30)
[NL]	'newline'	(Line 4, Col 31)
[DEDENT]	(generated)	(Line 5, Col 0)
[NL]	'newline'	(Line 5, Col 0)
[NAME]	'result'	(Line 6, Col 0)
[ASSIGN]	'='	(Line 6, Col 7)
[NAME]	'factorial'	(Line 6, Col 9)
[LPAREN]	'('	(Line 6, Col 18)
[NUM]	'5'	(Line 6, Col 19)
[RPAREN]	)'	(Line 6, Col 20)
[NL]	'newline'	(Line 6, Col 21)

[PRINT]	'print'	(Line 7, Col 0)
[LPAREN]	'('	(Line 7, Col 5)
[NAME]	'result'	(Line 7, Col 6)
[RPAREN]	')'	(Line 7, Col 12)
[EOF]	'<EOF>'	(Line 8, Col 0)

المرحلة 2: التحليل النحوي

Parse Tree (مبسطة):



AST الناتج:

```

ProgramNode
├── FunctionDefinitionNode [line 1]
│   ├── name: "factorial"
│   ├── parameters:
│   │   └── RegularParamNode: "n" [line 1]
│   └── body: BlockNode
│       ├── IfStatementNode [line 2]
│       │   ├── condition: ComparisonNode (<=) [line 2]
│       │   │   ├── left: VariableNode: "n"
│       │   │   └── right: NumberIntegerNode: 1
│       │   └── thenBlock: BlockNode
│       │       └── ReturnNode [line 3]
│       │           └── value: NumberIntegerNode: 1
│       └── ReturnNode [line 4]
│           └── value: BinaryOpNode (*) [line 4]
│               ├── left: VariableNode: "n"
│               └── right: FunctionCallNode: "factorial" [line 4]
│                   └── arguments:
│                       └── BinaryOpNode (-) [line 4]
│                           ├── left: VariableNode: "n"
│                           └── right: NumberIntegerNode: 1
├── AssignmentNode [line 6]
│   ├── target: "result"
│   └── value: FunctionCallNode: "factorial" [line 6]
│       └── arguments:
│           └── NumberIntegerNode: 5
└── PrintNode [line 7]
    └── expression: VariableNode: "result"
    
```

المرحلة 4: التحليل الدلالي

جدول الرموز:

Global Scope (level 0)				
Name	Type	Line	Col	Value
factorial	FUNCTION	1	4	-
result	VARIABLE	6	0	-
function_factorial (level 1)				
Name	Type	Line	Col	Value
n	PARAMETER	1	14	-

COMPILING (Python): factorial.py

Source Code:

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

result = factorial(5)
print(result)
```

Phase 1: Lexical Analysis...

[70 tokens generated]

Phase 2: Syntax Analysis (Parsing)...

[Parse tree built successfully]

Phase 3: Building AST...

[AST constructed]

Abstract Syntax Tree:

ProgramNode

FunctionDefinitionNode [line 1]

name: factorial

RegularParamNode: n [line 1]

BlockNode

IfStatementNode [line 2]

condition: ComparisonNode (<=)

VariableNode: n

NumberIntegerNode: 1

then:

ReturnNode

NumberIntegerNode: 1

ReturnNode

BinaryOpNode (*)

VariableNode: n

FunctionCallNode: factorial

BinaryOpNode (-)

VariableNode: n

NumberIntegerNode: 1

AssignmentNode [line 6]

target: result

FunctionCallNode: factorial

NumberIntegerNode: 5

PrintNode [line 7]

VariableNode: result

Symbol Table:

global (level 0)

Name	Type	Line	Col	Value
factorial	FUNCTION	1	4	-

result	VARIABLE	6	0	-
function_factorial (level 1)				
n	PARAMETER	1	14	-

✓ Compilation successful!

الملفات الأساسية

هنا أهم الملفات بالمشروع ووظيفة كل واحد:

جدول 10 : الملفات الأساسية

الوصف	المسار	الملف
نقطة الدخول الرئيسية - ينفذ المراحل الخمسة للترجمة	src/Main.java	Main.java
الكلاس الأساسي لكل عقد AST	src/antlr/ast/node/ASTNode.java	ASTNode.java
واجهة الـ Visitor Pattern	src/antlr/ast/visitor/ASTVisitor.java	ASTVisitor.java
بناء AST من Parse Tree (Python)	src/antlr/visitor/ASTBuilder.java	ASTBuilder.java
بناء AST من Parse Tree (Jinja2)	src/antlr/visitor/JinjaASTBuilder.java	JinjaASTBuilder.java
طباعة AST بشكل شجري ملون	src/antlr/visitor/ASTPrinter.java	ASTPrinter.java
جدول الرموز - تتبع المتغيرات والنطاقات	src/antlr/symbol/SymbolTable.java	SymbolTable.java
كلاس يمثل رمز واحد (متغير/دالة/etc)	src/antlr/symbol/Symbol.java	Symbol.java
قواعد المحلل المعجمي لـ Python	grammars/pythonLexer.g4	pythonLexer.g4
قواعد المحلل النحوي لـ Python	grammars/pythonParser.g4	pythonParser.g4
قواعد المحلل	grammars/jinja2Lexer.g4	jinja2Lexer.g4

		المعجمي لـ Jinja2 (modes 7)
jinja2Parser.g4	grammars/jinja2Parser.g4	قواعد المحلل النحوي لـ Jinja2
ProjectGenerator.java	src/antlr/generator/ProjectGenerator.java	منسّق المرحلة الثانية - توليد HTML من Flask
PythonEvaluator.java	src/antlr/generator/python/PythonEvaluator.java	تنفيذ كود إعداد البيانات والتقاط render_template
JinjaRenderer.java	src/antlr/generator/jinja/JinjaRenderer.java	عرض قالب Jinja2 إلى HTML
FlaskProjectAnalyzer.java	src/antlr/semantic/flask/FlaskProjectAnalyzer.java	فحص دلالي على مستوى المشروع
PythonSemanticAnalyzer.java	src/antlr/semantic/python/PythonSemanticAnalyzer.java	فحص دلالي لكود Python
JinjaSemanticAnalyzer.java	src/antlr/semantic/jinja/JinjaSemanticAnalyzer.java	فحص دلالي للقوالب
RTValue.java	src/antlr/runtime/values/RTValue.java	أساس نظام قيم الـ runtime
LiveServer.java	src/antlr/serve/LiveServer.java	الخادم المباشر
ReloadHub.java	src/antlr/serve/ReloadHub.java	WebSocket لإعادة تحميل المتصفح

جدول 11 : ملفات Python AST (مختارة)

الوصف	المسار	الملف
جذر البرنامج	src/antlr/ast/python/ProgramNode.java	ProgramNode.java
جملة إسناد	src/antlr/ast/python/statements/AssignmentNode.java	AssignmentNode.java
جملة شرطية	src/antlr/ast/python/statements/IfStatementNode.java	IfStatementNode.java
تعريف دالة	src/antlr/ast/python/statements/FunctionDefinitionNode.java	FunctionDefinitionNode.java
عملية ثنائية (+, -, /, *)	src/antlr/ast/python/expressions/operations/BinaryOpNode.java	BinaryOpNode.java
متغير	src/antlr/ast/python/expressions/access/VariableNode.java	VariableNode.java
رقم صحيح	src/antlr/ast/python/expressions/literals/NumberIntegerNode.java	NumberIntegerNode.java

جدول 12 : ملفات Jinja2 AST (مختارة)

الوصف	المسار	الملف
جذر القالب	src/antlr/ast/jinja2/TemplateNode.java	TemplateNode.java
حلقة for	src/antlr/ast/jinja2/blocks/ForBlockNode.java	ForBlockNode.java
شرط if	src/antlr/ast/jinja2/blocks/IfBlockNode.java	IfBlockNode.java
تعريف macro	src/antlr/ast/jinja2/blocks/MacroNode.java	MacroNode.java

HtmlElementNode.java	src/antlr/ast/jinja2/content/HtmlElementNode.java	عنصر HTML
DivHtmlNode.java	src/antlr/ast/jinja2/content/elements/sectioning/DivHtmlNode.java	عنصر div
ExpressionBlockNode.java	src/antlr/ast/jinja2/expressions/ExpressionBlockNode.java	كتلة تعبير { { }}

ملفات CSS AST (مختارة)

جدول 13 : ملفات CSS AST (مختارة)

الوصف	المسار	الملف
ورقة أنماط كاملة	src/antlr/ast/css/CSSStyleSheetNode.java	CSSStyleSheetNode.java
قاعدة CSS	src/antlr/ast/css/CSSRuleNode.java	CSSRuleNode.java
إعلان (property:) (value	src/antlr/ast/css/CSSDeclarationNode.java	CSSDeclarationNode.java
محدد class	src/antlr/ast/css/selectors/CSSClassSelectorNode.java	CSSClassSelectorNode.java
خاصية color	src/antlr/ast/css/properties/ColorPropertyNode.java	ColorPropertyNode.java

1. List Comprehensions (Python)

المشروع ببساطة list comprehensions المعقدة:

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

:AST

```
AssignmentNode
├── target: "squares"
└── ListComprehensionNode
    ├── expression: BinaryOpNode (**)
    │   ├── VariableNode: "x"
    │   └── NumberIntegerNode: 2
    ├── variable: "x"
    ├── iterable: FunctionCallNode: "range"
    │   └── NumberIntegerNode: 10
    └── condition: ComparisonNode (==)
        ├── BinaryOpNode (%)
        │   ├── VariableNode: "x"
        │   └── NumberIntegerNode: 2
        └── NumberIntegerNode: 0
```

2. Function Parameters المتنوعة

كل أنواع معاملات الدوال بـ Python مدعومة:

```
def complex_function(a, b=10, *args, x, y=20, **kwargs):
    pass
```

:Parameters

- RegularParamNode (positional) – a
- RegularParamNode – b=10 (مع قيمة افتراضية)
- StarArgsNode – *args
- KeywordOnlyParamNode – x (بعد *)
- KeywordOnlyParamNode – y=20 (مع قيمة افتراضية)
- KwargNode – **kwargs

3. Jinja2 بـ Whitespace Control

```
{%- if condition -%}
    content
{%endif -%}
```

ال - بيزيل المسافات البيضاء قبل/بعد ال tag.

Template Inheritance .4

```
{% extends "base.html" %}

{% block title %}My Page{% endblock %}

{% block content %}
    <p>Page content</p>
{% endblock %}
```

Macros مع معاملات .5

```
{% macro button(text, type="button", class="btn") %}
    <button type="{{ type }}" class="{{ class }}">{{ text }}</button>
{% endmacro %}

{{ button("Submit", type="submit", class="btn-primary") }}
```

Filters .6

```
{{ name | upper | trim }}
{{ items | length }}
{{ price | round(2) }}
```

Tests .7

```
{% if variable is defined %}
    {{ variable }}
{% endif %}

{% if value is none %}
    No value
{% endif %}
```

المرحلة الثانية: توليد الكود (Code Generation)

المترجم إلى مرحلتين. المرحلة الأولى (التي شرحناها فوق) بترجم ملف Python أو قالب Jinja2 لحالو وتُظهر الـ AST وجدول الرموز. أما المرحلة الثانية فبتحول مشروع ويب كامل مكتوب بـ Flask + Jinja2 إلى صفحات HTML ثابتة جاهزة للنشر — مع فحص دلالي متكامل قبل التوليد.

شو يعني "توليد كود"؟

الدخلة هي مشروع Flask عادي فيه:

- app.py - الكود الخلفي (البيانات + الـ routes)
- templates/ - قوالب Jinja2 مع HTML و CSS
- style.css و script.js (اختياري)

والخرجة هي صفحات HTML ثابتة - يعني القوالب انربطت بالبيانات الحقيقية من Python، فصارت صفحات جاهزة بدون ما تحتاج تشغيل السيرفر.

مثال - قبل التوليد (قالب Jinja2):

```
{% for product in products %}
<h3>{{ product.name }} - ${{ product.price }}</h3>
{% endfor %}
```

بعد التوليد (HTML ثابت):

```
<h3>Laptop - $999.99</h3>
<h3>Phone - $699.99</h3>
```

المخطط العام للمرحلة الثانية



↓
HTML ثابت → output/*.html

باختصار:

- Python → لتحضير البيانات فقط → تمريرها للقالب
- Jinja2 → لعرض البيانات → HTML

مراحل التوليد الخمسة

الكل بتنفيذها كلاس `ProjectGenerator.java`، وبكل مرحلة بتتسجل تفاصيلها بـ `generation_log.txt`.

المرحلة A: تحليل `app.py`

بنقرأ `app.py`، نمزرو على نفس خط أنابيب (AST → parse → lex) Python، ونعمل التحليل الدلالي لـ Flask:

```
PythonSemanticAnalyzer analyzer = new PythonSemanticAnalyzer();
analyzer.setSourceName("app.py");
analyzer.analyze(result.program);
result.symbolTable = analyzer.getSymbolTable();
```

الخرجات:

- `AST - compiler_output/ast_python.json` لملف Python بصيغة JSON
- `compiler_output/semantic_report.txt` - تقرير الدلالة

لو في أخطاء دلالية، بتتوقف عملية التوليد فوراً.

المرحلة B: تنفيذ كود إعداد البيانات

هون بننفذ الجزء اللي بجهز البيانات من كود Python (قوائم المنتجات، الحسابات...) بدون ما نشغل حقيقياً على الويب. الـ

`PythonEvaluator` بيمشي على الـ AST وبينتج سجل من قيم `runtime`، وبيمسك كل استدعاء `render_template(...)` ويحوّل

لطلب `RTRenderRequest` فيه اسم القالب + سياق المتغيرات.

```
@app.route('/')
def display_products():
    return render_template('products.html', products=products)
```

هي الاستدعائية بتتعامل كطلب توليد صفحة، بـ: القالب `products.html` والمتغير `products`. فبعدين بنجيب القالب ونربطو

بالبيانات.

المرحلة C: تحليل القوالب ودلالاتها

بنقرأ كل قالب مرجعي من `templates/`، ونبنيلو AST، ونحفظ كل القوالب بسجل مشترك (عشان دعم `{% include %}` و `{% extends %}`).

بعدها التحليل الدلالي على مستوى المشروع - `FlaskProjectAnalyzer` بيفحص:

- شو المتغيرات اللي القالب عم يستخدمها `{{ name }}`
- شو المتغيرات اللي `render_template` عم يوفرها
- لو في متغير عم يستخدم بالقالب وما في حدا عم يوفره → خطأ `'MISSING_TEMPLATE_VARIABLE'`

```
products.html:3:3 [MISSING_TEMPLATE_VARIABLE] - Semantic error:
template variable 'products' is not provided by render_template()
and is not defined in the template
```

هاي ميزة قوية: بتضمن إنو ما في قالب بيمرّي متغير غير موجود - بيكتشف المشكلة قبل التوليد.

- `AST - compiler_output/ast_jinja.json` لكل القوالب

المرحلة D: العرض (Rendering)

`JinjaRenderer` بيمشي على AST القالب، ويستبدل `{{ expression }}` بقيمتها الحقيقية من السياق، وبينفذ `{% for %}` و `{% if %}` و `{% include %}` ... لحتى ينتج HTML نهائي.

```
String html = renderer.render(req.template, req.request.getTemplateName(), context);
```

بعدها بكتب ملف HTML بـ `output/` بالاسم المناسب:

- صفحة ثابتة `output/products.html` →
 - صفحة ديناميكية (route مع معامل مثل `/product/5`) `output/product_5.html` →
- وبعدين `'StaticLinkRewriter'` بيصلّح الروابط: أي `href` أو `url_for` بيحوّلوا لمسار الملف الثابت الصحيح.

المرحلة E: الملفات المرافقة

الملفات اللي مش جزء من التحليل (بيتم نسخها كما هي بدون أي معالجة):

- `app.py` (نسخة للتشغيل)
- `style.css` و `script.js` (لو موجودة)

```

output/                                (ثابت HTML) الصفحات المولدة ←
├── products.html
├── product_1.html
├── product_2.html
├── product_3.html
└── app.py, style.css, script.js ← منسوخة كما هي

compiler_output/                       مخرجات التحليل والتوليد ←
├── ast_python.json
├── ast_jinja.json
├── semantic_report.txt
└── generation_log.txt

```

التوليد المتزايد (Incremental Generation)

بوضع `--watch --generate`، يمشي على تغييرات الملفات ويعيد التوليد بتكاه:

- لو تغيّر `app.py` → بيعيد كل المراحل
- لو تغيّر قالب → بيعيد تحليل وعرض الصفحات المتأثرة فقط

```

if (changedAbs.equals(appFileAbs)) {
    regenerateFromAppPy(startMs);    // كل المراحل
} else {
    regenerateFromTemplate(baseName, startMs); // القالب + العرض
}

```

قيم الـ Runtime

عشان ننفذ كود Python (المرحلة B) ونعرض القوالب (المرحلة D)، بنحتاج تمثيل للقيم أثناء التشغيل. هون بيحي دور `RTValue` -
نظام قيم موحد بيدعم كل أنواع Python:

```

RTValue (abstract)
├── RTInt      - أرقام صحيحة
├── RTFloat    - أرقام عشرية
├── RTBool     - true / false
├── RTString   - نصوص
├── RTList     - قوائم
├── RTDict     - (key → value) قاموس
├── RTFunction - دوال (لتنفيذ التعريفات والاستدعاءات)
├── RTNone     - none
└── RTUndefined - قيمة غير معرفة

```

ليش هاد النظام ضروري؟ لأنو المترجم بنفذ "سكربت" إعداد البيانات (اللي بجهز قوائم المنتجات مثلاً) لحتى يقدر يجيب القيم الحقيقية ويمزجها للقالب — مش بس يعرض الشجرة.

مثال بسيط:

```
// products = [...] → RTList يحتوي RTDict لكل منتج
RTList products = new RTList();
RTDict laptop = new RTDict();
laptop.set("name", new RTString("Laptop"));
laptop.set("price", new RTFloat(999.99));
products.add(laptop);
```

- الحسابات (ValueOps, PythonOperators) بتشتغل على هالقيم
- الاستدعاءات بتنفذ من FunctionInvoker
- الدوال المدمجة (print, max, range, ...float) موجودة بـ Builtins

التحليل الدلالي متعدد المستويات

المشروع إلو ثلاثة مستويات للفحص الدلالي، وكل مستوى بيكشف نوع معين من الأخطاء:

1. مستوى Python – PythonSemanticAnalyzer

بيفحص كود Python بدون ما ينفذو (نفس فئة أخطاء CPython):

جدول 14 : 1. مستوى Python – PythonSemanticAnalyzer

الكود	المعنى	مثال
UNDEFINED_VARIABLE	متغير مستخدم قبل تعريفه (NameError)	print(x) بدون x = ...
TYPE_ERROR	استدعاء/فهرسة شيء مش قابلو	products() مع قائمة products
TYPE_MISMATCH	أنواع معاملات غير متوافقة	5 - 'a' أو قائمة مفهرسة بنص
SCOPE_ERROR	return/break بغير مكانو	break خارج حلقة

ملاحظة ذكية: أي شيء بيعتمد على قيم وقت التشغيل بيتبقى ANY - يعني المحلل بيغلر بس الأخطاء اللي Python بترفضهن دايماً، وما بيخمن.

2. مستوى Jinja – JinjaSemanticAnalyzer

بيمشي على AST القالب ويتتبع المتغيرات والنطاقات داخل القالب (ماكرو، block، ...for).

3. مستوى المشروع – FlaskProjectAnalyzer

بينظم (...) render_template مع قوالبها ويكتشف متغيرات القالب غير المؤمنة (MISSING_TEMPLATE_VARIABLE) - شرحناه بالمرحلة C.

أمثلة على أخطاء دلالية مكتشفة

بالمجلد tests/error-demos/ في أمثلة جاهزة:

جدول 15 : أمثلة على أخطاء دلالية مكتشفة

المجلد	الخطأ المكتشف
undefined-variable/	متغير غير معرف
type-error/	استدعاء شيء مش قابل للاستدعاء
type-mismatch/	تعارض أنواع
scope-error/	return/break خارج مكانها
missing-flask-variable/	متغير قالب غير مؤمن من render_template

الخادم المباشر (Live Server)

الميزة الأخيرة هي وضع `serve` - بيقال flask --app app run --serve بيفتح سيرفر HTTP يعرض تطبيق Flask فعلياً على المتصفح، مع إعادة تحميل تلقائية عند أي تعديل.

كيف يشتغل؟

```
java Main serve (tests/flask/) سيرفر على منفذ عشوائي #
java Main serve 9000 tests/flask سيرفر على 9000 لمجلد محدد #
```

- يقرأ app.py وينبئه كتطبيق Flask حي (LiveApp)
- LiveServer يستمع لطلبات HTTP ويخدم الصفحات بالسيرفر الحقيقي
- ReloadHub (WebSocket) بيمكّن المتصفح من تحديث نفسه تلقائياً لما يتغير الملف

إعادة التحميل التلقائي

عند أي تعديل بملف:

- تغيير app.py → بيقراه من جديد ويعيد ضبط الحالة (state reset) + server.reload()
- تغيير قالب → بيحدّث القوالب (reloadTemplates()) + server.reload()

```
☑ Change detected: app.py
✓ app.py reloaded (state reset).
✓ templates reloaded.
```

المتصفح بيرجع يتحدث لحالو عبر WebSocket بدون أي refresh يدوي — نفس تجربة Flask debug=True بالضبط.

كيفية التشغيل

المتطلبات

- +Java JDK 11
- IntelliJ IDEA (أو أي Java IDE)
- ANTLR 4.13.2 (موجود بـ /dependencies)

خطوات التشغيل

1. فتح المشروع بـ IntelliJ IDEA

2. التأكد من إضافة ANTLR لا classpath:

- File → Project Structure → Libraries
- Add dependencies/antlr-4.13.2-complete.jar

3. Build المشروع:

- Build → Build Project

4. تشغيل Main.java:

① أوضاع المرحلة الأولى (تحليل وعرض):

```
# تشغيل كل الاختبارات
java Main

# تشغيل ملف محدد
java Main tests/dictionary.py
java Main tests/flask/templates/products.html

# إخفاء المسافات البيضاء بالعرض
java Main --hide-whitespace tests/flask/templates/products.html

# وضع المراقبة - إعادة الترجمة عند أي تعديل
java Main --watch
java Main --watch tests/flask
```

② توليد الكود - تحويل مشروع Flask إلى HTML ثابت:

```
# Flask من مشروع HTML توليد كل صفحات
java Main --generate tests/flask

# توليد +مراقبة (إعادة توليد تلقائية عند التعديل)
java Main --generate --watch tests/flask
```

③ الخادم المباشر (مثل `flask --app app run`):

```
# سيرفر على منفذ عشوائي
java Main serve

# سيرفر على منفذ محدد لمجلد معين
java Main serve 9000 tests/flask
```

شرح خيارات سطر الأوامر

جدول 16 : شرح خيارات سطر الأوامر

الخيار	الاختصار	الوظيفة
--generate [dir]	-g	المرحلة الثانية: توليد HTML من مشروع Flask
--watch [dir]	-W	مراقبة الملفات وإعادة الترجمة/التوليد تلقائياً
--hide-whitespace	-w	إخفاء عقد النص البيضاء بالعرض
serve [port] [dir]	-S	خادم مباشر مع تحديث تلقائي بالمتصفح

إعادة توليد الـ **Parsers** (لو عدلنا القواعد)

```
# Python
java -jar dependencies/antlr-4.13.2-complete.jar -visitor -listener \
  -package antlr.gen.python \
  -o src/antlr/gen/python \
  grammars/pythonLexer.g4 grammars/pythonParser.g4

# Jinja2
java -jar dependencies/antlr-4.13.2-complete.jar -visitor -listener \
  -package antlr.gen.jinja2 \
  -o src/antlr/gen/jinja2 \
  grammars/jinja2Lexer.g4 grammars/jinja2Parser.g4
```

ملفات الاختبار

جدول 17 : ملفات الاختبار

الوصف	الملف
اختبار شامل لكل ميزات Python - دوال، classes، list comprehensions	tests/dictionary.py
تعبير معقدة ومتداخلة	tests/teg/test_comprehensive.py
تطبيق Flask كامل	tests/flask/app.py
قالب Jinja2 لعرض منتجات	tests/flask/templates/products.html
تفاصيل منتج مع استايلات CSS	tests/flask/templates/product_detail.html

tests/flask/templates/add_product.html	نموذج إضافة منتج
tests/semantic/valid.py	كود Python صحيح يدعم الفحص الدلالي
tests/error-demos/undefined-variable/app.py	مثال خطأ: متغير غير معرف
tests/error-demos/type-error/app.py	مثال خطأ: استدعاء شيء غير قابل للاستدعاء
tests/error-demos/type-mismatch/app.py	مثال خطأ: تعارض أنواع
tests/error-demos/scope-error/app.py	مثال خطأ: break/return خارج مكانها
tests/error-demos/missing-flask-variable/app.py	مثال خطأ: متغير قالب غير مؤمن

الخلاصة

المرحلة الأولى - التحليل والفحص

- ✓ التحليل المعجمي - تقسيم النص لـ tokens مع دعم المسافات البادئة والأنماط المتعددة
- ✓ التحليل النحوي - بناء parse tree وكشف الأخطاء النحوية
- ✓ بناء AST - تحويل parse tree لشجرة مجردة باستخدام Visitor Pattern
- ✓ التحليل الدلالي - جدول رموز مع نطاقات متداخلة
- ✓ العرض - طباعة AST بشكل مقروء وملون

المرحلة الثانية - توليد الكود

- ✓ من Flask إلى HTML ثابت - تحويل مشروع ويب كامل لصفحات جاهزة
- ✓ تنفيذ إعداد البيانات - تنفيذ كود Python وتمير قيمه للقوالب
- ✓ فحص على مستوى المشروع - كشف متغيرات القوالب غير المؤمنة
- ✓ التوليد المتزايد - إعادة توليد ذكية عند التعديل
- ✓ خادم مباشر - تشغيل التطبيق مع تحديث تلقائي بالمتصفح عبر WebSocket
- ✓ دعم لغات متعددة - Python و Jinja2 مع HTML و CSS
- ✓ +136 عقدة AST - بنية شاملة ومنظمة